REVON: LOG AND MERKLE DIFFERENCING FOR DURABLE STRUCTURED DATA VERSIONING

BY ATHARVA SHEERSH PANDEY[1,a], ADHYAN JAIN[1,b], AND POORNIMA NEDUNCHEZHIAN[1,c]

1 School of Computer Science and Engineering, VIT Vellore

a atharva.sheersh2024@vitstudent.ac.in; b adhyan.jain2024@vitstudent.ac.in

c Corresponding author: Poornima Nedunchezhian (poornima.n@vit.ac.in)

Revon is a Python prototype for durable, linear-history key/value versioning. It supports log aggregation and hash-pruned comparison over the same SQLite-backed Merkle hash trie. We evaluate these two diff paths, an operation-count selection rule and the costs of complete versioning workflows. The Linux experiments comprise 516 executions and a separate 72-execution geometry and telemetry supplement. Eight of nine paired hybrid/Merkle timing intervals favour hybrid; the dense case includes parity. A separate threshold control selects Merkle when log is faster. The Windows evaluation covers 93 cases in fresh worker processes, with disjoint calibration and validation histories, controlled sensitivity experiments, Wine Quality and CO2 public sources, and repeated runs at 100,000 and 1,000,000 records. Its calibration winner uses log throughout the tested intervals, leaving the benefit of adaptive switching unresolved. CO2 histories capture observed publication changes, while wine corrections are generated. We verify diffs and historical states against semantic oracles and report algorithm, CLI-phase and whole-worker measurements, stating each boundary. The results describe the trade-offs observed on these machines and histories. They do not establish an optimal crossover, storage-engine superiority or reproduction by an independent team.

1. Introduction. A versioned store for structured data must recover past states and compare them when only part of the data has changed. Saving whole files addresses part of this requirement, but applications must still track and compare individual records. Git provides a familiar model: immutable objects are identified by their content and connected through parent commits [10]. Those objects describe files and directories. Record stores also need to handle persistent keys as update density and the number of commits between compared versions change.

We compare two ways of computing a diff over the same durable history. Log differencing aggregates recorded operations; Merkle differencing skips subtrees whose hashes match. Both paths operate on the same representation and produce the same semantic output. Revon combines established path copying, content addressing and SQLite transactions to let us compare these paths and a rule that selects between them. The evaluation concerns the measured versioning operations and workloads, rather than the cost of structured-data management as a whole.

Dolt combines SQL, a commit graph and Prolly Trees [13, 28]. ForkBase provides POS-Tree/SIRI storage with fork semantics [32], and Noms describes history-independent chunked collections [3]. Revon's interface is limited to key/value operations over a linear history. Its purpose here is to support a controlled diff-path comparison, without claiming

Keywords and phrases: data versioning, Merkle hash trie, hybrid differencing, content-addressed storage, benchmark methodology.

feature parity or a new family of persistent indexes. Cross-system measurements use Revon's in-process API and Dolt's CLI. We report the CLI phases separately to show the costs included in that comparison.

1.1. Research Questions. RQ1: How do complete versioning workflows trade latency against persistent storage and worker resources, and how do their API/CLI boundaries affect the interpretation of these measurements?

RQ2: On generated histories withheld from calibration, how does selection by operation count compare with forced log and forced Merkle over the same representation?

RQ3: When we control changes to the tested histories, how do workload structure, payload size and fixed-trie geometry affect the cost of differencing?

1.2. Contributions. Implementation contribution: Revon integrates content addressing, path copying and atomic SQLite persistence. The prototype exposes both diff paths through the same encoded-value contract, allowing us to compare them under matched conditions. This integration is the engineering contribution.

Policy contribution: we test a rule based on operation count against both forced diff paths. We complete calibration and record the chosen threshold before evaluating held-out histories. The analysis includes choices that make execution slower and asks whether using log throughout would suffice for these histories.

Evidence contribution: we check observed diffs and historical states against oracles built separately. The reproducibility materials retain the executed sources, raw measurements, environment records and data-preparation steps. They also document correctness checks and the limits of the telemetry.

Evaluation contribution: we report Linux workflow experiments and a 93-case Windows evaluation with separate calibration and validation seeds. The Windows cases include matched sensitivity controls, Wine Quality and CO2 sources, and repeated scale runs. We analyse each campaign using its own experimental units and keep its timing rows separate.

Our experiments cover a single-writer store, linear histories and bounded workloads. They show trade-offs within those conditions. They cannot identify a universally optimal selector or geometry, establish query-engine superiority, or demonstrate reproduction by an independent team.

2. Related Work.

2.1. Content Addressing and Persistent Structures. A Merkle root authenticates the collection stored below it [22]. Git applies content addressing to software history, and IPFS uses it for networked content [10, 5]. Path copying lets persistent structures retain earlier states without copying the entire structure [14]. Disk-oriented dictionaries include ordered stratified B-trees [31] and fully persistent B-trees [8]. Revon's fixed-depth trie routes keys by hash. Equal hashes allow a diff traversal to stop, but this routing cannot provide ordered range access. SciDB and TimeArr use layouts and deltas designed for versioned arrays [27, 30]. TardisDB supports version access in a main-memory database, and Saquib et al. study replicated versioned structures for wide-area systems [25, 26]. These studies explain several relevant design choices; they do not measure Revon's log and Merkle paths over a common history.

Merkle Search Trees combine search-tree ordering with Merkle authentication to reconcile states [4]. Revon also uses hashes to prune comparisons, but has fixed-depth routing and a centralized linear history rather than a replicated CRDT. LBFS uses content-defined chunking to find redundant content [23]; Dolt and Noms use related methods to

determine chunk boundaries. Such boundaries can preserve deduplication as data shifts, while ordered indexes support range operations. Neither capability is evaluated by the Revon benchmark.

2.2. Structured Dataset Versioning. Dataset versioning involves collaboration as well as the cost of storing and recovering states. DataHub studies collaborative histories at scale [6]. Decibel supports relational branching and compares version-first, tuple-first and hybrid layouts [20]. OrpheusDB partitions data to improve version retrieval when versioning is added to an existing DBMS [19]. Bhattacherjee et al. formalize the trade-off between storage and reconstruction [7]. Delta Lake provides ACID tables through transaction logs and checkpoints [2]. Version-graph methods choose materialized states and deltas subject to storage and retrieval constraints [18]. The FAIR-compliant proposal addresses consistent release identification and description [16]; it complements storage research without defining a diff algorithm. Revon's key/value interface covers only part of the wider task of managing datasets.

2.3. System Design Gap. Noms represents structured values in a Merkle DAG and supports diff and synchronization [3]. ForkBase integrates content addressing with fork and deduplication semantics [32]. Dolt uses content-addressed Prolly Trees for structural sharing, native diff and ordered indexing [13, 28]. Git4Data offers snapshot, branch, diff and merge operations through SQL over a relational engine [17]. We use its available arXiv preprint to describe the abstraction, without treating the reported results as peer-reviewed or independently reproduced evidence. Revon has a narrower implementation: fixed-depth hash routing in a single-writer key/value store with linear history, without SQL or a branching interface.

Table 1 brings together the systems and studies most relevant to these implementation choices.

Table 1
Related-work summary

System or study	Representation or focus	Relevance to Revon
Persistent DS [14]	Path copying	Formal structural-sharing basis
Git [10]	Object DAG	Identity and commit ancestry; file-oriented
IPFS [5]	Merkle DAG	General content-addressed versioning
MST [4]	Merkle search tree	Authenticated ordered reconciliation
DataHub [6]	Version graph	Collaborative dataset motivation
Decibel [20]	Relational layouts	Native database branching
OrpheusDB [19]	Partitioned versions	Storage/retrieval optimization
Delta Lake [2]	Log/checkpoints	ACID time travel at data-lake scale
Noms [3]	Merkle DAG	Structured content addressing
ForkBase [32]	Forkable CAS	Fork and deduplication semantics
DoltHub [13, 28]	Prolly Trees	Mature production comparator

2.4. Design Implications and Research Positioning. The choice of representation determines when a version history incurs the work of creation, storage, reconstruction and comparison. Complete snapshots simplify retrieval but copy records that have not changed. Logs leave work for replay or checkpointing. Path copying reuses unchanged structure and creates the paths affected by an update [14, 7]. Decibel and OrpheusDB examine how layout affects version creation and retrieval [20, 19], while version-graph analyses explicitly relate storage to retrieval cost [18]. Reproducibility research considers how to preserve builds and artifacts for later inspection [1]. We record import, commit, checkout, diff and storage separately for these reasons. Preserving artifacts makes inspection possible; it does not demonstrate faster execution.

Stored values capture only one meaning of change. Differential dataflow updates query results when timestamped inputs change [21]. Provenance describes the origins of output facts and why they occur [9]. DSLog records fine-grained array lineage and supports queries on it [33]. Explain-Da-V connects semantic changes in datasets to transformations [29], and Baseline uses operation-based differencing to handle refactorings and schema changes [15]. Revon identifies keys added, removed or modified between two states. It does not infer their causes, lineage, effects on queries or intended schema changes. These studies help place that task in context, but differences in their outputs and workloads prevent direct timing comparisons with Revon's key/value diffs.

A Revon-H commit retains both a Merkle root and a content-addressed changeset. Either path can recover the same changes between endpoints, so the selection rule chooses how to compute them. We hypothesize that log aggregation suits short, sparse intervals and that hash pruning may help after longer operation sequences. Whether their costs cross is an empirical question. Git4Data illustrates the broader interest in database-native version control [17]. Here, Snapshot and Log-only provide representation baselines for a fixed-depth-trie comparison. The public workloads use generated corrections to TLC and wine records, and observed changes in a CO2 publication sequence. These sources do not establish performance across independent hardware, concurrent workloads or production systems.

2.5. Capabilities and comparison boundaries. Table 2 distinguishes the systems' abstractions from the evidence used in this paper. Its capability descriptions are not benchmark results. ForkBase and Noms document typed-collection APIs; we do not evaluate a relational layer over them. Git4Data is represented by a preprint at a different abstraction level [17]. An unevaluated capability should not be read as an unsupported capability.

Table 2
System capabilities and the scope of the evidence; documented features do not establish performance

System	Representation / history	Diff / persistence	Branches / queries	Reproducibility and use here
Revon	Fixed-depth hash trie; linear history	Log or Merkle; SQLite objects	No named branches/merge; key/value API	Executed snapshots, semantic proofs and raw trials; measured here
Dolt [13, 28]	Prolly Trees; commit graph	Native version diff; persistent block store	Branches/merge; SQL	Official docs/source; CLI workflow measured, engine not isolated
ForkBase [32]	POS-Tree/SIRI; forkable versions	Hash-pruned diff/merge; addressed chunks	Fork on demand/conflict; typed KV APIs	Primary paper and APIs; no experiment run here
Noms [3]	History-independent chunks; Prolly collections	Hash-based diff/sync/merge; chunk store	Typed collection API; not benchmarked here	Technical overview/source; conceptual comparison
Git4Data [17]	MatrixOne immutable objects/MVCC; table versions	Database-native diff/merge	Snapshot/tag/branch ; SQL extensions	Preprint/BranchBench claims belong to its authors; not run here

2.6. Related foundations and scope boundaries.

Research-data versioning guidance treats version identity and change description as part of reliable citation and reuse [34]. The IPFS evaluation and FastVer integrity system study content-addressed persistence and verification in settings broader than Revon's local prototype [35, 36]. Persistent tries, B+-trees, external-memory search trees, snapshot stores and authenticated data structures provide different ways to retain and verify earlier states [37-41]. Work on repository views, collaborative analytics and DEX shows how history systems support repository access and collaboration [42-44].

Schema-evolution and temporal-database research examines changes in data structure and meaning over time [45-47]. Provenance research describes where results originate and how they are derived. This work includes provenance semirings, surveys of scientific workflows, tutorials and broader syntheses [48-51]. Other studies cover aggregate-query provenance, authenticated or secure provenance, FAIR data stewardship, privacy and the PROV model [52-56]. SEEK illustrates the integration of provenance and data management in a scientific repository [57].

Tree and log designs have a long history in storage research. Foundational work on B-trees and cache-oblivious indexing examines different search and update costs [58-61]. LSM trees, Bigtable, Dynamo and log-structured file systems are influential examples of storage organized around logs or appends [62-65]. WiscKey and LogBase study write-efficient key-value storage and log-based databases, while multi-version partitioned B-trees address indexing across versions [66-68]. These designs inform our comparisons, but their workloads and measurements cannot substitute for those evaluated here.

3. Revon Design.

Figure 1 presents Revon's implemented architecture and identifies the operations evaluated here.

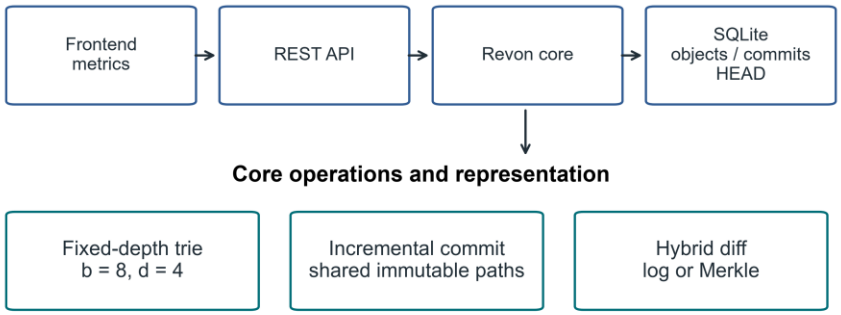


Fig 1. Revon's architecture and the scope of the implemented research prototype

3.1. Addressed Object Model. Revon writes trie nodes and changesets as canonical JSON, using the SHA-256 digest of each object as its name. For a commit, the digest covers the root, parent, changeset, message and timestamp. Human-readable labels such as v1 and v2 are aliases, so they do not affect object identity. The durable HEAD reference points to the current commit.

3.2. Fixed-Depth Merkle Hash Trie. We use b=8 and d=4 for the main configuration. At each level, routing reads three bits of SHA-256(key). The resulting twelve bits select one of 4,096 possible leaf buckets. Leaves retain full keys, so keys with a shared routing prefix remain distinguishable. Expected occupancy is N/b^d under uniform hashing. We evaluate other b,d combinations separately rather than assuming that this default minimizes cost.

3.3. Incremental Hashing and Commit. For a mutation batch, Revon canonicalizes the values and groups keys that share a routing prefix. It rewrites each affected leaf once, then rebuilds the ancestors along the affected paths. Internal nodes retain unchanged child hashes, allowing the corresponding structure to be shared. A single SQLite transaction writes the objects, changeset, commit metadata, version alias and measurements, and updates HEAD. Reopening a repository verifies stored payload hashes and references.

3.4. Hybrid Diff. Log differencing walks the requested ancestor interval and combines operations by key. Merkle differencing compares roots recursively, stopping at equal hashes and examining entries below unequal paths. Revon-H first validates the version identifiers, checks ancestry in both directions and counts changeset operations. An eligible ancestor interval uses log when the count is no greater than T; otherwise it uses Merkle. For reverse log requests, old and new values exchange places. Requests comparing a version with itself return an empty diff. Forced log rejects unrelated versions, while unknown identifiers raise an error. Revon-M always requests Merkle, although its public version-pair API also checks ancestry. The two paths produce the same sorted semantic diff. We time commit and checkout independently.

The Linux workflow protocol fixes T=4,096. A separate Linux control uses T=16,384, selected in an auxiliary calibration experiment. T counts operations, while b^d determines bucket capacity. We fit neither parameter to the Linux results.

3.5. JSON value ownership and identity. Revon accepts JSON nulls, booleans, finite numbers, strings, lists and objects whose keys are strings. Before storage, it validates input containers and detaches them from the caller. Reads and diagnostic views also return detached containers; immutable scalars may be shared. Writes reject unsupported Python values, nested non-string keys, non-finite numbers and cyclic containers. Equality uses the v1 JSON encoding, with sorted object keys, compact separators, UTF-8 and no NaN. This preserves distinct encodings for true, 1, 1.0 and signed floating zero. These rules define the prototype's Python encoding contract without claiming compliance with a cross-language canonical-JSON standard.

Algorithm 1 (batch update). First validate the batch and take ownership of its values. Before interning objects, reject any key that appears in both puts and deletes, as well as requests to delete missing keys. Read the old values and keep only mutations that change encoded identity. Group these mutations by routing prefix. Rewrite each affected leaf, sort its entries and hash the full payload. Rebuild the affected ancestors, reusing unchanged child hashes where possible, and then record the content-addressed changeset and commit. Detaching the values prevents callers from changing stored history through input containers or supported reads.

Algorithm 2 (log composition). Walk the ancestor interval in chronological order. For each key, retain its existence and value at the first old endpoint and at the last new endpoint. Discard the key only if existence and encoded value agree at both endpoints. An insertion followed by deletion cancels; an update followed by restoration does too. A null value remains distinct from an absent key. Sort the keys that remain. For a reverse request, exchange the endpoints and swap the added/deleted labels.

Algorithm 3 (selection). Validate the pair of versions and check whether either is an ancestor of the other. Add the changeset operation counts across the interval. Use log if the interval is eligible and its count is at most T; otherwise use Merkle. Merkle traversal skips subtrees with equal hashes and compares values in unequal leaves. Both paths follow the same encoded-identity rule and return detached values.

Cost model. Let N denote the record count, d the depth, b the branching factor, m the number of keys in a batch, h the interval length, p the accumulated operations, u the distinct keys in that interval and s the serialized output bytes. Uniform routing gives expected leaf occupancy N/(b^d), although worst-case occupancy is N. Processing a batch requires input validation and copying, m routed lookups of old values, and scans of occupied leaves. The touched leaves must be sorted and serialized, and the affected internal child maps copied. Batch processing therefore has no dataset-independent O(d) bound.

Log composition walks ancestry and takes O(p + u log u) work, plus the cost of comparing encoded values and copying/serializing the output. For Merkle, the cost depends on the node pairs visited, the sorting of child slots and the leaf entries examined. Value comparison and output sorting add further work. Both public version-pair modes walk ancestry; in a reverse request, the unsuccessful direction may first reach genesis. Constructing the output takes at least work proportional to s. Large JSON values also incur comparison and copying costs that scalar-only benchmarks leave unmeasured.

When Revon opens SQLite storage, it eagerly loads the retained nodes, changesets and commits. The objects persist on disk but remain resident during execution, so retained history must fit in memory. The retained payload also affects repository opening and verification. The prototype has no disk-page cache or eviction policy. SQLite transaction atomicity supplies the persistence mechanism. Establishing production concurrency and operational guarantees would require further tests.

Figure 2 traces path selection for a version pair, covering eligibility checks and reverse requests.

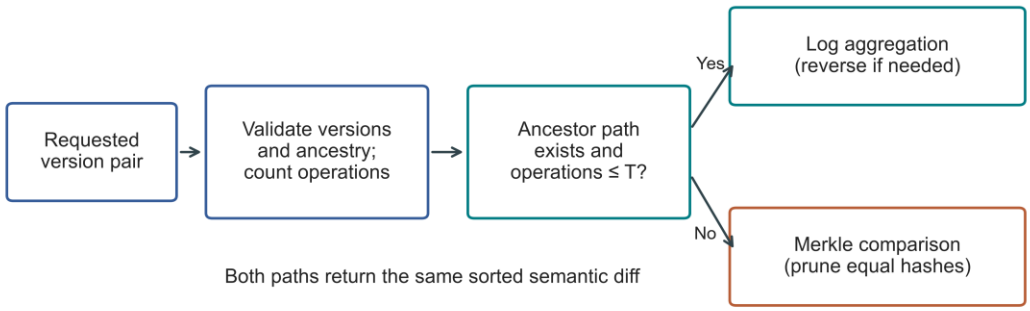


Fig 2. Choosing a diff path for a version pair. T denotes the operation threshold. Invalid versions cause an error before selection. Both paths operate on the same persistent trie

4. Experimental Methodology.

4.1. Environment and evidence boundaries. We ran the Linux experiments on an Ubuntu n2-standard-4 cloud VM. Table 3 lists the environment and the executed sources. The Windows experiments used a separate computer operated by the authors. Hardware, OS, runtime and storage all differ between the campaigns, so their effects cannot be separated. Both campaigns were conducted by the authors.

Table 3
Linux execution environment and source records

Property	Recorded configuration
Machine	GCP n2-standard-4; Intel Cascade Lake; 4 vCPUs; 16 GB RAM; us-east1-c
Operating system	Ubuntu 24.04.5 LTS; x86_64; kernel 7.0.0-1011-gcp
Storage	100 GB balanced persistent disk; no swap configured
Runtime	CPython 3.14.3, Clang 22.1.1 build; GIL enabled; JIT not enabled; SQLite 3.50.4
Comparator and packages	Dolt 2.3.1; psutil 7.1.0; PyArrow 25.0.1
Primary source	linux-validation-20261008: executed source, fixed before measurement
Supplement source	linux-final-supplement-20261008: same core; zero-aware I/O sampler and geometry history checks
Background conditions	Google Ops Agent active; filesystem cache not cleared; monitoring included

Each campaign records its executed source hashes, fixed protocol, environment, correctness checks and raw observations. The Linux preflight suites passed 67 and 69 tests; the Windows suite passed 77. The records include setup failures and the public-data exclusion made before timing. Appendices A and B explain access and give the detailed procedures.

4.2. Variants and workloads. The five systems provide six workflows. Snapshot stores a full state at each version; Log-only stores an initial state and subsequent operations. Revon-M requests Merkle diff, while Revon-H chooses a path. Dolt holds records in a primary-key SQL table and produces its native JSON diff. We measure SQL INSERT and CSV table import as separate Dolt workflows. Although the systems differ in interfaces and storage features, their subsequent operations follow the same contract for semantic output. Table 4 defines the workload IDs used in the figures.

Table 4
Linux workloads with ten generated commits in every primary history

ID / scenario	Rows	Changes / commit	Locality
S1 Small sparse	1,000	10	Spread
S2 Medium sparse	10,000	10	Spread
S3 Medium dense	10,000	1,000	Spread
S4 Large sparse	100,000	100	Spread
S5 Application-key local	100,000	100	Lexically adjacent keys
S6 Range local	100,000	100	Contiguous key intervals
S7 Repeated key	100,000	100	Repeated small key set
S8 Hash-route local	100,000	100	Shared 9-bit hash prefix
P1 TLC public records	100,000	100	Generated fare corrections

Each of the eight synthetic scenarios runs six variants with two warm-ups and seven measured trials, giving 432 executions. Public-record workloads add 27 executions for Revon-M, Revon-H and Dolt SQL. Threshold controls add 54. The three million-row diagnostics bring the total to 516, comprising 114 warm-ups, 399 measured trials and three diagnostics. All executions completed and their outputs were verified. Failed timing executions are neither silently replaced nor left out of this accounting.

4.3. Public dataset and generated histories. For TLC, we use the January 2024 yellow-taxi release [24]. Its checksum identifies a Parquet input containing 2,964,624 rows. A dispersed affine permutation, seeded with 20261001, produces nested subsets of 10,000, 100,000 and 1,000,000 rows. We keep eleven fields: vendor, pickup/drop-off times, passenger count, distance, pickup/drop-off locations, payment type, fare, tip and total. Each key combines the release, file-checksum prefix and physical row ordinal. It identifies a record in that file without asserting trip identity across releases. We encode values, including nulls, as compact UTF-8 JSON with sorted keys. All rows serialize successfully, including the 47,277 missing passenger counts in the million-row subset. Rows with identical field values remain separate. The affine sampling procedure does not give a uniformly random sample.

We generate ten-commit TLC histories. A commit changes 100 records in the Linux 100,000-row workload and 1,000 records in the million-row diagnostic. Each correction adds 0.01 to fare and total and leaves the other retained fields as they were. Inputs and

mutations are identical across systems. These edits are generated rather than observed trip revisions. The public-data builder ignores the configuration's inherited payload_bytes=8 field and retains all eleven source fields; measured serialized bytes determine payload size. We store each canonical record as a JSON-encoded string in the common value column. These measurements therefore cover neither native mutation of nested objects nor queries on individual fields.

4.4. Timing, correctness and uncertainty. We prepare the workload, construct the repository, time its operations, verify correctness and then analyse the results. Import, commit, diff and checkout each have their own timing boundaries. Under the shared output contract, a diff returns sorted keys with hashes of the old/new values. Table 5 sets out the boundaries for algorithm work, interface costs and whole-worker measurements. Correctness checks and decision probes run outside the operation timers.

Table 5
Timing and interpretation boundaries

Quantity	Revon / same-representation control	Dolt CLI workflow	Interpretation
Diff latency	Version API, selected path and canonical output	Native JSON-diff command, parsing and canonical output	Complete interface contract; no engine-only ranking
Phase probe	Separate ancestry/count probe; includes detached-view copying	Command, JSON parse and output-construction timers	Neither probe nor command isolates pure tree-engine work
Correctness	All states, reverse/identity, integrity and reopen outside timers	Observed native rows and historical checkout outside timers	Observed values compared with an independently constructed oracle
Resources	Worker plus sampled children, including oracle/setup	Worker plus sampled Dolt processes	10 ms samples; short processes/peaks may be missed
Persistent bytes	Windows: closed files; Linux: before close/compaction	Repository directory files	Includes metadata/indexes; not isolated algorithm overhead

We verify the observed endpoint diff and every historical state against workload oracles built separately from the storage implementation. Revon also checks reverse and identity requests, the integrity of stored objects, and reopening. The audits reconstruct the plan and semantic hashes to establish internal consistency. They do not rerun the timing experiment.

For each fixed Linux history, we estimate timings from seven trials, each with a fresh repository, and calculate paired percentile-bootstrap intervals. Windows cases make seven cached queries within a fresh repository. We separately report their medians across generated histories or repetitions of a repository. The experimental units thus differ, and neither campaign samples a population of machines.

Sampling follows the worker and its observed descendants through setup and verification as well as storage operations. CPU and memory measurements therefore include work beyond the storage implementation. In the primary Linux records, read counters do not distinguish supported zero values from missing counters. The Linux supplement and the Windows sampler explicitly distinguish them. Windows process I/O bytes still do not identify direct physical-disk traffic.

Repository size includes persistent metadata and the cost of the representation. We record Linux directory size before closing or compacting, but Windows size after closing

and the specified verification steps. Because these conditions differ, we report the measurements separately. Neither measurement isolates tree overhead.

4.5. Frozen threshold and geometry controls. An auxiliary Windows calibration chose T=16,384, its largest candidate, without locating a crossover. We use that threshold unchanged in the Linux controls. The Windows selector experiment described below has a separate fixed design and executed source, with disjoint calibration/validation seeds. We analyse its threshold results separately.

Over 10,000 rows, the Linux control compares forced log, forced Merkle and hybrid with T=16,384 at accumulated operation counts of 8,192 and 32,768. Windows outcomes do not determine or retune this control.

The Linux geometry experiment uses five configurations on one fixed spread workload. Windows sensitivity cases use three generated histories and matched equal-capacity geometries. Bucket capacity and the operation threshold are separate parameters. These experiments characterize the settings tested; they do not identify a geometry that is generally optimal.

4.6. Windows calibration and validation. The Windows selector comparison fixes N=12,000, H=8, payload=32 bytes and b=8,d=4. We cross spread/repeated-key histories with 64, 512 and 4,096 changes per commit, using two calibration seeds and three different validation seeds. Forced log, forced Merkle and hybrid thresholds T=512/4,096/16,384/65,536 all use the same durable representation within a case. Configuration order is deterministically shuffled. Two warm-ups come before seven measured query blocks. Each case starts in a fresh worker, but its queries share resident objects and filesystem cache, so the design does not measure a cold cache.

During calibration, we divide each hybrid case median by the lower of the forced-path medians. We choose the threshold with the lowest equally weighted mean across cases, breaking exact ties in favour of the smaller T. We record the threshold and its selection time in UTC before the first validation worker starts. Validation cannot choose a new winner. Sensitivity cases vary rows, history interval, payload, geometry and routing according to a staged design specified in advance. At fixed accumulated operations, a longer interval means fewer changes per commit, which prevents a fully factorial design. The appendices and protocols list every seed and matched control.

4.7. Public histories and repeated scale experiments. Wine Quality contains 6,497 red/white records [11]; their keys combine colour and source row ordinal. We generate three histories, each with eight batches of 64 seeded corrections to the quality field. These corrections are generated, rather than observed revisions to specimens. The CO2 package provides published NOAA-derived CSV revisions [12]. Before freezing the experiment, we collect the eleven most recent snapshots in which the file changed and exclude one publication failure containing only a header. Ten populated snapshots yield nine observed transitions, starting from 812 records. Unnamed trailing columns are retained as extra fields. Dates identify records; CSV ordering and formatting do not enter the state comparison. This is an observed repository publication history, not NOAA transaction logs or evidence about atmospheric science.

The three CO2 repositories repeat one observed sequence. Wine and scale repositories instead use distinct generated seeds. Each public source is measured through Revon-M, T=4,096 Revon-H and native Dolt JSON diff, with model order rotated across runs. Scale experiments fix H=4, C=64 and payload=32 at 100,000 and 1,000,000 records, using three seeds per size/system. We report medians and ranges across repository medians

descriptively. The metadata lists source URLs, licences, acquisition versions, checksums and transformations. Limiting acquisition to a bounded publication history let us collect the source on the available host without a bulk Software Heritage collection.

Appendix B provides the execution-count matrix in Table 17 and the TLC checksums and Linux public-data protocol in Table 16. The main text states the timing boundaries and experimental units so that the results can be interpreted without reconstructing the full execution manifest.

5. Linux Results.

5.1. Hybrid versus forced Merkle. Revon-M and Revon-H store the same persistent structures and changesets. This ablation therefore varies only the requested diff strategy. Figure 3 and Table 6 report all nine comparisons with repeated trials. Eight hybrid intervals exclude parity, while the dense S3 interval includes one. Log access reduces diff time for the sparse and repeated-key histories tested. It does not reduce the stored representation, and these results do not show that switching benefits every workload.

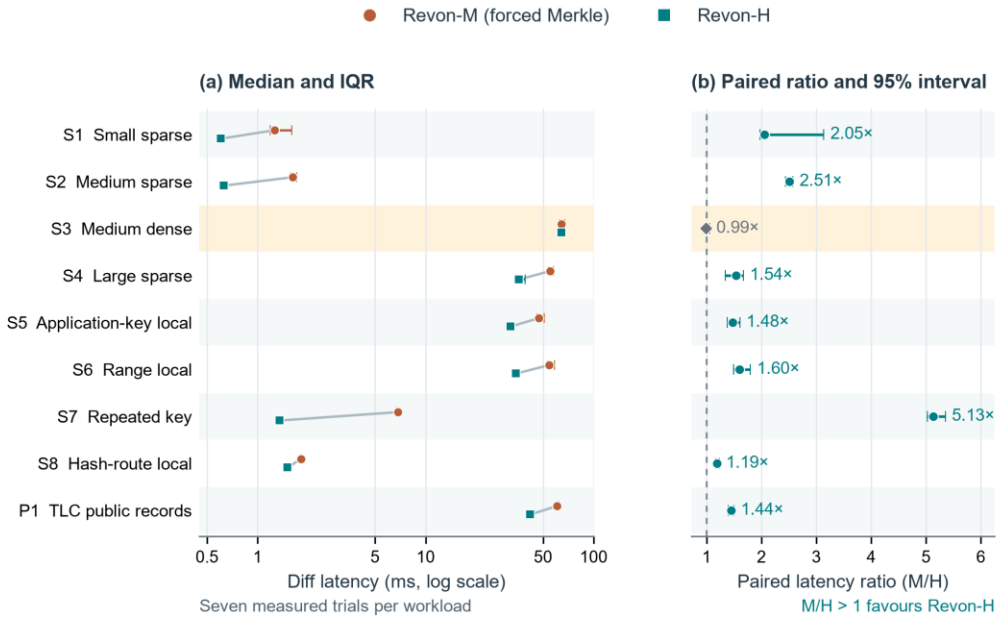


Fig 3. Linux diff timings for Revon-M and Revon-H. Panel (a) plots median latency and interquartile ranges on a logarithmic axis, connecting each workload pair. Panel (b) plots the median matched-trial M/H ratio with paired percentile-bootstrap 95% intervals based on 20,000 resamples. The dashed line denotes parity. There are seven measured trials per workload. We highlight S3 because its interval includes parity; the other eight favour H. The median of trial ratios can differ from the ratio of latency medians. Workload definitions appear in Table 4. These intervals reflect repeated timings of fixed histories on one VM. They do not describe variation across independent datasets or machines.

Table 6
Paired M/H diff ablation: latency in milliseconds and median trial ratio [95% interval]

ID	M (ms)	H (ms)	M/H ratio [interval]	H path
S1	1.259	0.604	2.05 [1.97, 3.14]	log
S2	1.620	0.629	2.51 [2.44, 2.57]	log
S3	64.135	64.183	0.99 [0.97, 1.06]	merkle
S4	54.897	35.749	1.54 [1.34, 1.67]	log
S5	47.253	31.967	1.48 [1.38, 1.60]	log
S6	54.352	34.537	1.60 [1.50, 1.80]	log
S7	6.862	1.345	5.13 [5.02, 5.35]	log
S8	1.817	1.503	1.19 [1.17, 1.23]	log
P1	60.597	41.964	1.44 [1.39, 1.50]	log

On P1, Revon-H has a median diff latency of 41.964 ms, compared with 60.597 ms for forced Merkle. Both use identical eleven-field public records and generated edits. For S3, H selects Merkle because the interval contains 10,000 accumulated operations. Near-parity is consistent with both variants taking the same diff path, with H also performing selection.

5.2. Threshold control and adverse result. Figure 4 and Table 7 include both controls and the case where selection is adverse. Hybrid chooses log at 8,192 operations and Merkle at 32,768. At the larger count, forced log is faster in every one of the seven matched measured trials. Operation count alone therefore fails to select the fastest path in these cases. The control tests the fixed choice on Linux without resolving which threshold is best.

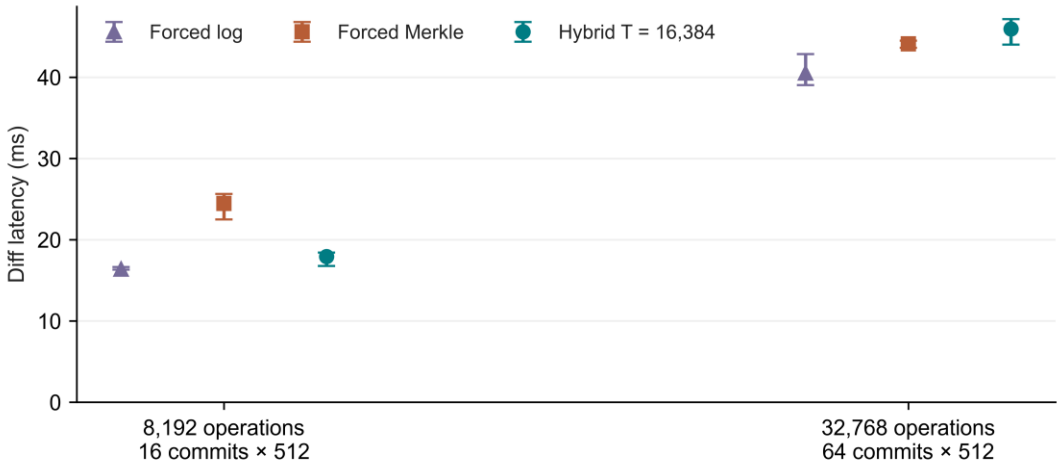


Fig 4. Linux control with the fixed T=16,384 policy. Points mark medians and bars mark IQRs for seven measured trials per path. Labels identify the history lengths

Table 7
Frozen threshold controls; diff medians in milliseconds

Operations / history	Forced log	Forced Merkle	Hybrid	Selected path
8,192 / H=16	16.364	24.466	17.926	log
32,768 / H=64	40.520	44.120	45.944	merkle

5.3. Complete workflow comparison. Table 8 compares the complete workflows producing a canonical diff through Revon-H's API and Dolt's SQL/CLI interface. Each of the nine paired Dolt/H intervals is above one. Timed work includes data structures, process startup, language, serialization, query execution and resident state. The ratios consequently cannot isolate storage-engine performance. Figure 5 separates commit from checkout to show how the costs change with the operation.

Table 8
Dolt SQL/CLI and Revon-H diff comparison: latency in milliseconds and paired ratio [95% interval]

ID	Dolt (ms)	H (ms)	Dolt/H ratio [interval]
S1	90.704	0.604	148.15 [139.77, 160.15]
S2	95.515	0.629	148.70 [140.08, 150.80]
S3	297.617	64.183	4.70 [4.54, 4.77]
S4	152.472	35.749	4.25 [3.84, 4.34]
S5	125.594	31.967	3.73 [3.71, 3.87]
S6	135.765	34.537	3.88 [3.69, 4.12]
S7	97.451	1.345	71.73 [68.95, 74.32]
S8	110.966	1.503	71.24 [68.83, 77.06]
P1	168.576	41.964	4.02 [3.77, 4.07]

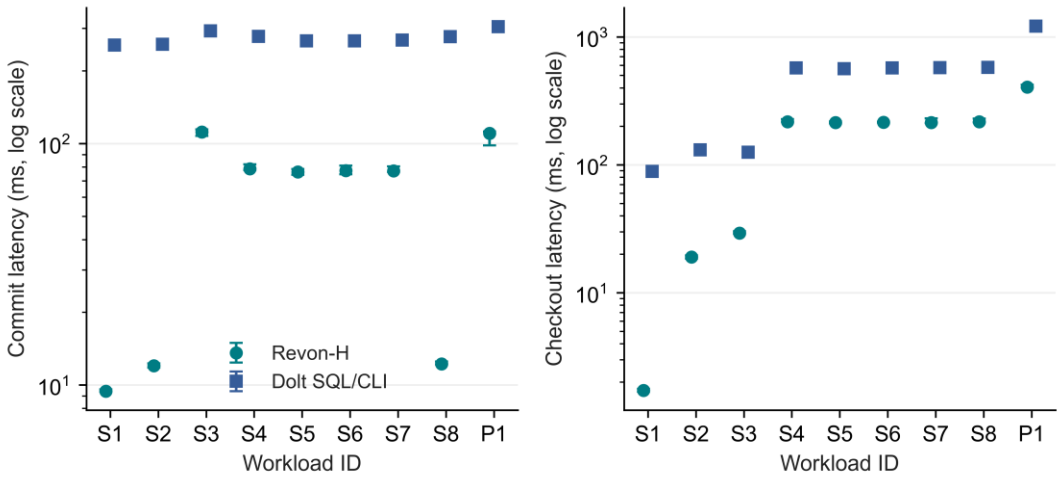


Fig 5. Linux latency for incremental commits and full checkout through Revon-H and Dolt SQL/CLI. Points mark medians and bars mark IQRs across seven trials. The latency axes use logarithmic scales

5.4. Storage and memory costs. Figure 6 and Table 9 place persistent directory size and peak RSS for the complete worker alongside the latency results. Both M and H store content-addressed changesets and trie data. The small difference in directory size comes from the measured workflows; selecting log removes no stored data. Dolt also uses different serialization, compression and indexing. Faster diffs can therefore come with substantially larger repositories. Assessing efficiency requires considering storage and worker resources alongside latency.

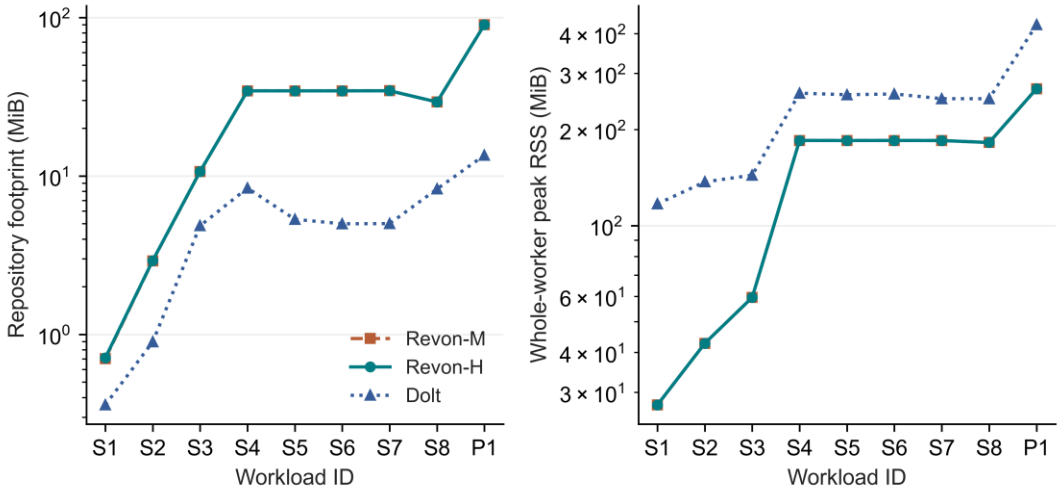


Fig 6. Median repository size before compaction and median peak RSS for the whole worker across seven Linux trials, plotted on logarithmic axes. RSS includes memory used for the shared workload and verification

Table 9
Repository footprint and worker RSS; medians in MiB

ID	M disk	H disk	Dolt disk	H RSS	Dolt RSS
S1	0.7	0.7	0.4	27.4	116.9
S2	2.9	2.9	0.9	42.7	137.1
S3	10.7	10.7	4.8	59.5	143.9
S4	34.4	34.4	8.4	184.9	260.1
S5	34.4	34.4	5.3	184.8	257.2
S6	34.4	34.4	5.0	184.9	258.6
S7	34.5	34.5	5.0	184.8	250.0
S8	29.3	29.3	8.3	182.2	249.9
P1	90.1	90.0	13.5	268.6	426.5

5.5. Million-record feasibility. In the Linux million-row diagnostic, we create and verify one fresh repository each for M, H and Dolt. Verification covers all eleven historical states and the shared diff oracle. Table 10 gives the individual observations without IQRs, confidence intervals or a ranking. Hybrid uses Merkle because the 10,000 accumulated operations exceed T=4,096. These observations show that the bounded workload completes on the VM. One observation per system does not establish relative scaling or an optimal configuration. Section 6.4 reports the separate repeated Windows scale experiment.

Table 10
Single-run million-record diagnostics; n=1 per system, no comparative inference

System	Import (s)	Commit (ms)	Diff (ms)	Checkout (s)	RSS (GiB)
Dolt	24.902	547.311	817.034	11.454	2.555
Revon-H	32.289	2961.901	1233.601	4.729	2.561
Revon-M	32.454	2979.717	1265.367	4.685	2.561

5.6. Fixed-trie geometry and telemetry. Figure 7 and Table 11 compare the Linux geometry configurations. The b4-d6, b8-d4 and b16-d3 settings each provide 4,096 leaf buckets. Equal expected occupancy lets us examine routing shape at that capacity. With b=8 fixed, changing depth also changes capacity and the balance between traversal and leaf scanning. Because this experiment uses one fixed workload on one VM, it cannot identify a geometry that is optimal in general.

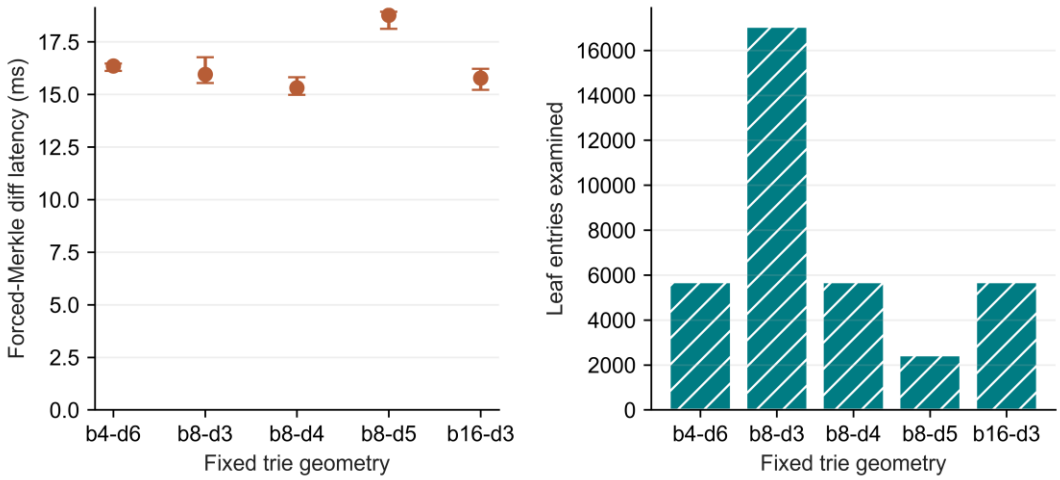


Fig 7. Fixed-trie geometry on Linux. The left panel plots forced-Merkle diff medians and IQRs across seven trials. The right panel plots the median number of examined leaf entries

Table 11
Geometry work and footprint; one fixed spread workload

Geometry	Buckets	Diff (ms)	Leaf entries	Node pairs	Disk (MiB)
b4-d6	4096	16.355	5682	3628	4.32
b8-d3	512	15.949	17048	585	6.88
b8-d4	4096	15.316	5682	3729	4.25
b8-d5	32768	18.765	2426	6089	6.18
b16-d3	4096	15.775	5682	3929	8.12

Two isolated-worker telemetry tests distinguish valid all-zero I/O samples from unavailable counters, and both pass. The Linux telemetry supplement uses 27 executions on a fixed 10,000-row workload: M, H and Dolt each contribute two warm-ups and seven measured trials. The records explicitly retain zero measured read bytes rather than leaving blanks. This confirms how the counters handle zero, but does not establish cold-storage behaviour or exclude missed samples. We analyse the primary latency summaries separately.

5.7. Representation baselines. Table 12 gives descriptive medians for the six workflows on S4, a representative spread workload. The GitHub reproducibility archive in Appendix A provides the full eight-scenario matrix, trial dispersion, normalized storage and separate SQL/CSV import workflows. Snapshot and Log-only act as representation controls in the shared harness, although they do not match every production feature. Their full-state storage and operation-replay costs help explain Revon's measurements.

Table 12
Workflow medians for the representative S4 case; the GitHub reproducibility archive contains the full eight-scenario matrix

Workflow	Import (s)	Commit (ms)	Diff (ms)	Checkout (ms)	Disk (MiB)
Snapshot	0.071	69.577	624.860	168.158	53.5
Log-only	0.072	2.474	16.483	159.538	5.0
Revon-M	1.488	78.842	54.897	215.661	34.4
Revon-H	1.644	78.590	35.749	217.482	34.4
Dolt SQL	1.663	277.593	152.472	572.585	8.4
Dolt CSV	1.335	273.425	149.325	559.342	7.9

6. Windows Evaluation Results.

6.1. Held-out selector behaviour. Calibration chooses T=65,536, the largest candidate, with a normalized score of 1.014. Its mean normalized latency across 18 held-out histories is 1.012; T=4,096 gives 1.101. The thresholds agree with the lower forced-path median in 18/18 and 12/18 cases, respectively. Those counts compare choices with noisy measured medians, rather than measuring classification accuracy against a known optimum. Table 13 and Figure 8 show all profile groups.

No primary Windows interval exceeds 32,768 operations, so T=65,536 chooses log throughout this matrix. Winning calibration thus shows neither a benefit from switching nor a global crossover. Timing variation can produce ratios below one even when hybrid and forced execution choose equivalent paths. This result and the adverse Linux control constrain the conclusions we can draw about selection. The experimentally chosen threshold leaves the implementation's default unchanged.

Table 13
Held-out results as the median of three history medians, with fixed N/H/payload and no population interval

Locality	Operations	Forced log ms	Forced Merkle ms	H4096 ms	Calibrated ms
spread	512	1.913	4.622	1.907	1.837
spread	4,096	25.714	38.878	26.550	27.312
spread	32,768	92.792	104.71	106.56	91.775
repeated-key	512	0.564	1.661	0.549	0.573
repeated-key	4,096	5.608	14.342	6.311	4.664
repeated-key	32,768	62.290	74.366	77.092	67.721

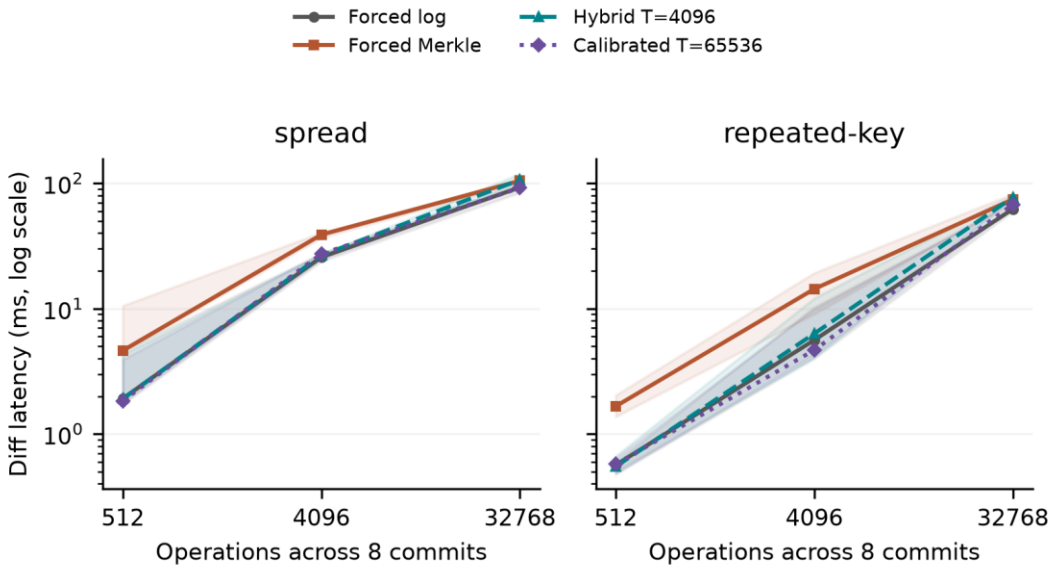


Fig 8. Held-out selector results at N=12,000 and H=8. Lines join the medians of three generated-history medians. Shading covers their minimum to maximum, and each history contributes seven measured cached queries. This shading describes the observed range; it is not a 95% confidence interval. At every plotted operation count, the calibrated policy chooses log. All forced paths share the same durable representation.

6.2. Sensitivity and resource interpretation. The decision probe reads a detached public changeset view outside diff timing. Copying that view adds cost, so the probe cannot isolate selector overhead. To examine this separately, a diagnostic extracts the prefix of the production function that validates versions, checks ancestry and counts operations internally, stopping before dispatch. Across the 18 held-out histories and two thresholds, 252,000 measured warm calls yield 2.674 microseconds as the median of the case/threshold medians, with a range of 1.611-3.678. The diagnostic makes the same decisions as production dispatch. It excludes both diff traversal and the copying of detached views. It is not a cold-cache estimate or a causal correction to subtract from workflow timings. The diagnostic is separate from the 93-case workflow campaign.

The 21 sensitivity cases vary a prespecified factor or matched control across three seeds. They cover payload=256 bytes, N=24,000, fixed-operation H=16, equal-capacity b4-d6/b16-d3 geometries and a C=8 hash-route/spread pair. Geometry cases reproduce the base history's state hashes while changing only routing geometry. Holding operations fixed makes history length trade off against changes per commit, so that comparison does not identify their independent effects. The results describe these profiles rather than selecting a generally best geometry.

Median forced-Merkle latencies were 47.857 ms for more rows, 40.122 ms for longer histories, 39.950 ms for larger payloads, 43.004 ms for b4-d6 and 38.500 ms for b16-d3 geometries, and 0.174 ms for hash-route locality compared with 0.903 ms for its matched spread control. Each number is the median of three history medians. The summaries retain the timing for each history, the paths selected and the scope of the resource measurements. These exploratory comparisons make no claim of significance after multiple testing.

For T=4,096, the three sampler off/on comparisons give median-latency ratios ranging from 0.950 to 1.206. Within each pair, sampling-off runs come first. We control neither cache state nor background activity. The ratios therefore cannot identify a causal sampling penalty, and we subtract no correction from other timings. Resource counters measure the

entire worker. Tests retain observed zero I/O as a value, but Windows counters cannot identify database logical reads or direct physical-disk traffic.

6.3. Public data and interface phases.

Table 14
Wine and CO2 workflows: median and range of three repository medians; CO2 repeats one observed history

Source / system	Runs	Diff ms	Range ms	Import s	Persistent MiB
co2/revon-m	3	0.612	0.562-0.661	0.033	1.191
co2/revon-h	3	0.309	0.302-0.318	0.035	1.184
co2/dolt	3	256.98	251.87-262.89	0.634	0.178
wine/revon-m	3	4.281	4.260-4.964	0.189	10.832
wine/revon-h	3	2.247	1.956-2.366	0.191	10.812
wine/dolt	3	296.76	246.75-302.81	0.819	1.360

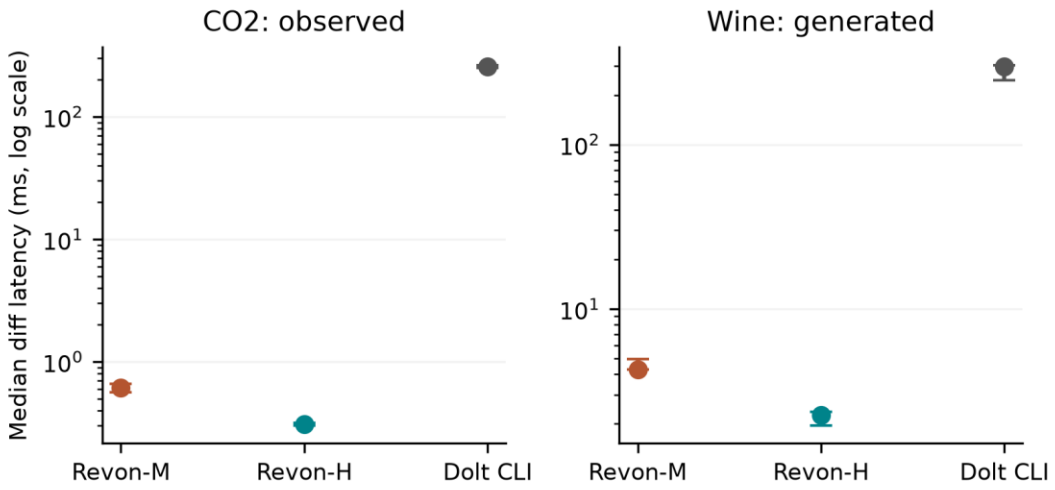


Fig 9. Public-source workflows. Each point marks the median of three repository medians, and each bar spans their full range. The CO2 repositories repeat the same observed publication history. The wine repositories instead use three generated correction histories. Each repository supplies seven cached queries. Because the API/CLI boundaries differ, these timings describe complete workflows rather than isolated storage engines.

For co2-observed, median Dolt command time is 256.063 ms, JSON interpretation 0.501 ms and common output construction 0.503 ms. The command accounts for 99.6% of median complete diff latency. That time combines startup, native work and native JSON emission. The breakdown cannot attribute the entire interval to the storage engine or justify removing a startup constant measured separately.

For wine-generated, the median Dolt command takes 288.464 ms, with 4.445 ms for JSON interpretation and 3.258 ms for common output construction. Command time accounts for 97.2% of median complete diff latency and includes startup, native work and native JSON emission. As with CO2, the measurements do not isolate engine cost or support subtracting an independent startup estimate.

6.4. Repeated Windows scale observations.

Table 15
Repeated Windows scale study: medians of three seed/repository summaries; full ranges in evidence

Rows / system	Import s	Commit ms	Diff ms	Checkout s	Worker RSS GiB
100,000/revon-m	1.116	54.309	5.649	0.209	0.248
100,000/revon-h	1.109	53.044	1.004	0.173	0.242
100,000/dolt	1.662	759.99	283.65	0.662	0.223
1,000,000/revon-m	10.614	88.233	33.694	1.862	1.939
1,000,000/revon-h	10.738	85.362	1.013	1.575	1.925
1,000,000/dolt	11.281	890.97	320.30	3.706	0.976

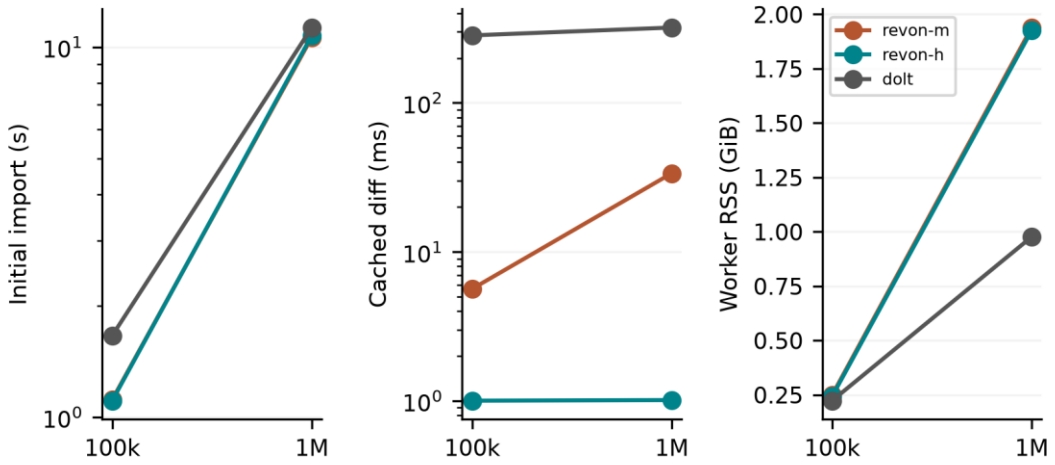


Fig 10. Windows scale observations from three generated histories and fresh repositories per size/system. Lines connect descriptive medians at two sizes; they are not fitted asymptotic laws. Diff queries use cached state, while import and checkout are timed separately. Peak RSS includes workload/oracle generation, storage work, verification and sampled descendants. The Linux million-row n=1 diagnostics use a different campaign.

We analyse the repeated Windows scale runs separately from the Linux diagnostics. The three histories start from the same synthetic distribution and apply different updates; they do not represent independent public datasets. The manifest records available memory and background processes, which we do not control. These measurements describe the workflows under the recorded conditions. They cannot establish a population ranking or a general scaling law.

6.5. Answers to the research questions. For RQ1, complete workflows reveal trade-offs between latency, persistent storage and worker resources. Their API/CLI boundaries prevent ranking the engines alone. For RQ2, log improves timings on several fixed histories, but the Windows calibration winner uses it in every tested case, leaving the benefit of adaptive switching unestablished. For RQ3, the controlled histories show effects of payload, routing and geometry within the staged design without identifying a universally best layout. Observed CO2 transitions, generated wine corrections and synthetic updates supply different kinds of history, each with its own limits.

7. Discussion. The workflow results answer RQ1 at the interface an application would use. On both Linux and Windows, we measure import, commit, diff, checkout and persistent footprint, and analyse the campaigns separately. Snapshot and Log-only expose the costs of their representations. Comparisons of forced M, H and log hold Revon's representation constant. Dolt offers a richer SQL/versioning interface. Breaking down its command phases helps readers inspect the workflow, but does not equate its native tree work with an isolated Python call.

Answering RQ2 requires testing whether selection transfers beyond the histories used to choose the parameter. The Windows split fixes primary N, H, payload and geometry, and records selection before validation. The largest candidate acts as always-log throughout the measured range. Both the adverse Linux threshold choice and parity in the dense case show that the rule cannot be presumed to choose the fastest path. Any more complex selector would need its own calibration and untouched test data. These findings provide no basis for changing the default threshold.

For RQ3, equal capacity does not imply equal traversal depth or ancestor-reconstruction cost, even when uniform routing gives the same expected occupancy. Encoding payloads, examining touched leaves and constructing output also take time. Windows cases use separate generated seeds and matched controls; the Linux geometry experiment uses one fixed history. Neither design estimates all interactions or identifies an optimum across a population.

Diff comparisons depend on consistent value semantics. Revon detaches inputs and outputs to prevent mutable aliases from changing stored history. Its v1 encoded identities distinguish true from 1 and 1 from 1.0, without changing the object encoding or on-disk schema. The Linux trials exercise these rules on the supplied histories. Unit tests also cover null versus absence, nested ownership, unsupported values and rollback.

8. Limitations And Future Work. External validity: we ran Linux experiments on one cloud VM and Windows experiments on one local host. Neither campaign is an independent-party replication or a controlled comparison of operating systems. Hardware, runtime, virtualization and storage all change together. Filesystem caches and background services remain active. The Windows manifest also records limited available RAM and the process conditions. These measurements do not describe an entirely idle host or a cold cache.

Workload validity: the TLC histories apply generated updates to one monthly release. Linux scale diagnostics have n=1 per system. Windows validation includes three held-out generated seeds per primary profile, generated wine corrections and one observed CO2 publication sequence. Its three CO2 repositories replay that sequence rather than sample independent histories. We exclude a header-only revision before timing and retain unnamed columns without interpreting their scientific meaning. Synthetic scale histories begin with the same distribution. Generalizing further requires additional domains, observed histories and hardware.

Implementation scope: the trie has fixed geometry and eagerly loads retained nodes, changesets and commits, with no eviction. Revon supports a single writer and linear history. This evaluation does not cover named branches, merge commits, conflict handling, schema evolution, remotes, ordered scans, crash fault injection or concurrent workloads. Atomic SQLite transactions by themselves do not demonstrate the full durability and operational guarantees required in production.

Measurement and inference: Dolt command time includes process startup, native work and JSON emission. Worker RSS also includes oracle and history state. Sampling every ten milliseconds can miss brief processes or peaks. Windows I/O counters identify neither

physical-disk traffic nor database logical reads. The sampler off/on comparisons also confound order with cache conditions. The Linux bootstrap intervals concern repeated timings of fixed histories. Windows ranges across three histories are descriptive and have no multiple-comparison correction. Checks of sources and semantic hashes establish internal consistency without independently reproducing timings.

Selection and design: primary Windows calibration tests four thresholds on two generated seeds and the specified profiles. The largest candidate, T=65,536, never switches within the tested range. The broader value of adaptive selection therefore remains unestablished. Sensitivity testing uses a staged design rather than a fully factorial one: at fixed operation count, H trades off against density, and hash-route cases require their matched low-density control. Testing a different selector or establishing a crossover requires a separately fixed experiment.

Binary nodes, value chunks, packed objects, compression and reachability-based garbage collection are possible extensions to storage. An ordered Merkle index or content-defined chunking could change the design of range operations. Each extension would need separate semantic and performance tests, since results for the fixed trie cannot predict its effects.

9. Conclusion. Revon implements two diff paths with consistent semantics over a content-addressed fixed-trie history stored atomically in SQLite. We contribute an inspectable prototype and measurements of its workflow and selection costs. The Linux experiments include 516 executions and a separate 72-execution supplement, with uncertainty in the dense case and an adverse threshold choice. On Windows, 93 worker cases check 3,402 queries across held-out histories, Wine Quality and CO2 sources, interface phases and repeated scale runs. Calibration chooses T=65,536, which uses log throughout the tested primary matrix. We therefore leave the optimal crossover unresolved. The recorded workloads show conditional trade-offs subject to the stated API/CLI, sampling, dataset and hardware limits. They establish neither a universal speed advantage nor reproduction by an independent party.

Funding: We received no funding for this work.

Competing interests: We declare no competing interests.

Ethics and consent to participate: We recruited no participants and carried out no interventions involving humans or animals. This computational study uses synthetic histories and publicly released de-identified TLC records. We claim neither institutional ethics approval nor an exemption.

Consent for publication: No identifiable participant material appears in the paper, so consent for publication is not applicable.

Data and code availability: We use public NYC TLC taxi records [24], UCI Wine Quality [11] and the published CO2 package [12]. Code, executed source snapshots and experimental evidence are available at https://github.com/atharvasheersh/Revon. The paper/ directory holds the Linux reproducibility archive and the Windows reproducibility archive. We do not redistribute TLC input records or generated databases. The source metadata lists licences and transformations; Appendix A describes access.

Acknowledgment. We thank the open-source Dolt project and the researchers whose work on dataset versioning informed the experimental design.

REFERENCES

[1] M. Akhlaghi et al. (2021). Toward long-term and archivable reproducibility, Computing in Science & Engineering, vol. 23, no. 3, pp. 82-91, https://doi.org/10.1109/MCSE.2021.3072860. [Online]. Available: https://arxiv.org/abs/2006.03018.

[2] M. Armbrust et al. (2020). Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores, Proc. VLDB Endowment, vol. 13, no. 12, pp. 3411-3424, https://doi.org/10.14778/3415478.3415560.

[3] Attic Labs. Noms Technical Overview. GitHub. [Online]. Available: https://github.com/attic-labs/noms/blob/master/doc/intro.md.

[4] A. Auvolat and F. Taïani (2019). Merkle Search Trees: Efficient State-Based CRDTs in Open Networks, in Proc. 38th IEEE Int. Symp. Reliable Distributed Syst. (SRDS), pp. 221-230, https://doi.org/10.1109/SRDS47363.2019.00032.

[5] J. Benet (2014). IPFS: Content Addressed, Versioned, P2P File System, arXiv:1407.3561, [Online]. Available: https://arxiv.org/abs/1407.3561.

[6] A. Bhardwaj et al. (2015). DataHub: Collaborative Data Science and Dataset Version Management at Scale, in Proc. 7th Biennial Conf. Innovative Data Syst. Research (CIDR), Open copy: https://www.cidrdb.org/cidr2015/Papers/CIDR15_Paper18.pdf.

[7] S. Bhattacherjee, A. Chavan, S. Huang, A. Deshpande, and A. Parameswaran (2015). Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff, Proc. VLDB Endowment, vol. 8, no. 12, pp. 1346-1357, https://doi.org/10.14778/2824032.2824035.

[8] G. S. Brodal, S. Sioutas, K. Tsakalidis, and K. Tsichlas (2020). Fully persistent B-trees, Theoretical Computer Science, vol. 841, pp. 10-26, https://doi.org/10.1016/j.tcs.2020.06.027. [Online]. Available: https://cs.au.dk/~gerth/papers/tcs20.pdf.

[9] P. Buneman, S. Khanna, and W.-C. Tan (2001). Why and where: A characterization of data provenance, in Database Theory: ICDT Lecture Notes in Computer Science, vol. 1973, 2001, pp. 316-330, https://doi.org/10.1007/3-540-44503-X_20. [Online]. Available: https://www.pure.ed.ac.uk/ws/files/16509989/Why_and_Where_A_Characterization_of_Data_Provenance.pdf.

[10] S. Chacon and B. Straub (2014). Git Internals: Git Objects, in Pro Git, 2nd ed., [Online]. Available: https://git-scm.com/book/en/v2/Git-Internals-Git-Objects.

[11] P. Cortez, A. Cerdeira, F. Almeida, T. Matos and J. Reis (2009). Wine Quality [Dataset], UCI Machine Learning Repository, https://doi.org/10.24432/C56S3T. CC BY 4.0.

[12] DataHub Contributors (2026). CO2 PPM - Trends in Atmospheric Carbon Dioxide, NOAA-derived published data package, accessed October https://github.com/datasets/co2-ppm. ODC-PDDL-1.0; acquired revision identifiers and file hashes are retained in the experiment manifest.

[13] DoltHub (2026). Dolt Storage Engine: Block Store, Dolt Documentation, accessed Oct. 2, [Online]. Available: https://www.dolthub.com/docs/architecture/storage-engine/block-store/.

[14] J. R. Driscoll, N. Sarnak, D. D. Sleator, and R. E. Tarjan (1989). Making data structures persistent, J. Comput. Syst. Sci., vol. 38, no. 1, pp. 86-124, https://doi.org/10.1016/0022-0000(89)90034-2.

[15] J. Edwards and T. Petricek (2025). Baseline: Operation-based evolution and versioning of data, arXiv:2512.09762, preprint, updated 2026. [Online]. Available: https://arxiv.org/abs/2512.09762.

[16] A. González-Cebrián, M. Bradford, A. E. Chis, and H. González-Vélez (2024). Standardised versioning of datasets: A FAIR-compliant proposal, Scientific Data, vol. 11, art. 358, https://doi.org/10.1038/s41597-024-03153-y. [Online]. Available: https://www.nature.com/articles/s41597-024-03153-y.

[17] H. Gou, Z. Zhang, Y. Sun, P. Xu, F. Tian, L. Wang, and J. Wang (2026). Git4Data: Database-native version control for AI agents, arXiv:2609.02106, preprint. [Online]. Available: https://arxiv.org/abs/2609.02106.

[18] A. Guo, J. Li, P. Sukprasert, S. Khuller, A. Deshpande, and K. Mukherjee (2024). To store or not to store: A graph theoretical approach for dataset versioning, in Proc. IEEE Int. Parallel and Distributed Processing Symposium (IPDPS), pp. 479-493, https://doi.org/10.1109/IPDPS57955.2024.00049. [Online]. Available: https://arxiv.org/abs/2402.11741.

[19] S. Huang, L. Xu, J. Liu, A. J. Elmore, and A. Parameswaran (2017). OrpheusDB: Bolt-on Versioning for Relational Databases, Proc. VLDB Endowment, vol. 10, no. 10, pp. 1130-1141, https://doi.org/10.14778/3115404.3115417.

[20] M. Maddox et al. (2016). Decibel: The Relational Dataset Branching System, Proc. VLDB Endowment, vol. 9, no. 9, pp. 624-635, https://doi.org/10.14778/2947618.2947619.

[21] F. McSherry, D. G. Murray, R. Isaacs, and M. Isard (2013). Differential dataflow, in Proc. 6th Biennial Conf. Innovative Data Systems Research (CIDR), [Online]. Available: https://www.cidrdb.org/cidr2013/Papers/CIDR13_Paper111.pdf.

[22] R. C. Merkle (1988). A digital signature based on a conventional encryption function, in Advances in Cryptology: CRYPTO '87, Lecture Notes in Computer Science, vol. 293, pp. 369-378, https://doi.org/10.1007/3-540-48184-2_32.

[23] A. Muthitacharoen, B. Chen, and D. Mazières (2001). A low-bandwidth network file system, in Proc. 18th ACM Symp. Operating Syst. Principles (SOSP), pp. 174-187, https://doi.org/10.1145/502034.502052.

[24] New York City Taxi and Limousine Commission (2024). TLC Trip Record Data: Yellow Taxi, January public dataset, accessed Oct. 1, 2026. [Online]. Available: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page.

[25] N. Saquib, C. Krintz, and R. Wolski (2023). Replicated versioned data structures for wide-area distributed systems, IEEE Transactions on Parallel and Distributed Systems, vol. 34, no. 1, pp. 207-224, https://doi.org/10.1109/TPDS.2022.3217969. [Online]. Available: https://racelab.cs.ucsb.edu/papers/tpds22-versioned.pdf.

[26] M. E. Schüle, L. Karnowski, J. Schmeißer, B. Kleiner, A. Kemper, and T. Neumann (2019). Versioning in main-memory database systems: From MusaeusDB to TardisDB, in Proc. 31st Int. Conf. Scientific and Statistical Database Management (SSDBM), pp. 169-180, https://doi.org/10.1145/3335783.3335792. [Online]. Available: https://www-db.cs.tum.edu/~schuele/data/tardis.pdf.

[27] A. Seering, P. Cudré-Mauroux, S. Madden, and M. Stonebraker (2012). Efficient versioning for scientific array databases, in Proc. IEEE 28th Int. Conf. Data Engineering (ICDE), pp. 1013-1024, https://doi.org/10.1109/ICDE.2012.102. [Online]. Available: https://dspace.mit.edu/entities/publication/a5c2c050-6abd-47c1-8bee-4f8c51b6a15f.

[28] T. Sehn (2024). Dolt's Storage Engine, DoltHub Blog, Feb. 29, [Online]. Available: https://www.dolthub.com/blog/2024-02-29-storage-engine/.

[29] R. Shraga and R. J. Miller (2023). Explaining Dataset Changes for Semantic Data Versioning with Explain-Da-V, Proc. VLDB Endow., vol. 16, no. 6, pp. 1587-1600, https://doi.org/10.14778/3583140.3583169.

[30] E. Soroush and M. Balazinska (2013). Time travel in a scientific array database, in Proc. IEEE 29th Int. Conf. Data Engineering (ICDE), pp. 98-109, https://doi.org/10.1109/ICDE.2013.6544817. [Online]. Available: https://scidb.cs.washington.edu/paper/ICDE13_conf_full_422.pdf.

[31] A. Twigg, A. Byde, G. Miłos, T. Moreton, J. Wilkes, and T. Wilkie (2011). Stratified B-trees and versioning dictionaries, in Proc. 3rd USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage), [Online]. Available: https://arxiv.org/abs/1103.4282.

[32] S. Wang et al. (2018). ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications, Proc. VLDB Endowment, vol. 11, no. 10, pp. 1137-1150, https://doi.org/10.14778/3231751.3231762.

[33] J. Zhao and S. Krishnan (2024). Compression and in-situ query processing for fine-grained array lineage, arXiv:2405.17701, [Online]. Available: https://arxiv.org/abs/2405.17701.

[34] J. Klump, H. Pampel, L. Rothfritz, and D. Strecker (2024). Recommendations on Data Versioning. Zenodo. https://doi.org/10.5281/zenodo.13743875

[35] D. Trautwein, A. Raman, G. Tyson, I. Castro, W. Scott, M. Schubotz, et al. (2022). Design and evaluation of IPFS. Proceedings of the ACM SIGCOMM 2022 Conference, pp. 739-752. https://doi.org/10.1145/3544216.3544232

[36] A. Arasu, B. Chandramouli, J. Gehrke, E. Ghosh, D. Kossmann, J. Protzenko, et al. (2021). FastVer: Making Data Integrity a Commodity. Proceedings of the 2021 International Conference on Management of Data, pp. 89-101. https://doi.org/10.1145/3448016.3457312

[37] E.D. Demaine, S. Langerman, E. Price (2010). Confluently Persistent Tries for Efficient Version Control. Algorithmica, vol. 57, no. 3, pp. 462-483. https://doi.org/10.1007/S00453-008-9274-Z

[38] S. Lanka, E. Mays (1991). Fully persistent B+-trees. ACM SIGMOD Record, vol. 20, no. 2, pp. 426-435. https://doi.org/10.1145/119995.115861

[39] G.S. Brodal, C.M. Rysgaard, R. Svenning (2023). External Memory Fully Persistent Search Trees. Proceedings of the 55th Annual ACM Symposium on Theory of Computing, pp. 1410-1423. https://doi.org/10.1145/3564246.3585140

[40] C. Allain, B. Clément, A. Moine, G. Scherer (2024). Snapshottable Stores. Proceedings of the ACM on Programming Languages, vol. 8, no. ICFP, pp. 338-369. https://doi.org/10.1145/3674637

[41] A. Miller, M. Hicks, J. Katz, E. Shi (2014). Authenticated data structures, generically. Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, pp. 411-423. https://doi.org/10.1145/2535838.2535851

[42] V. Salis, D. Spinellis (2019). RepoFS: File system view of Git repositories. SoftwareX, vol. 9, pp. 288-292. https://doi.org/10.1016/j.softx.2019.03.007

[43] A. Bhardwaj, A. Deshpande, A.J. Elmore, D. Karger, S. Madden, A. Parameswaran, et al. (2015). Collaborative data analytics with DataHub. Proceedings of the VLDB Endowment, vol. 8, no. 12, pp. 1916-1919. https://doi.org/10.14778/2824032.2824100

[44] A. Chavan, A. Deshpande (2017). DEX. Proceedings of the 2017 ACM International Conference on Management of Data, pp. 171-186. https://doi.org/10.1145/3035918.3064056

[45] J.F. Roddick (1995). A survey of schema versioning issues for database systems. Information and Software Technology, vol. 37, no. 7, pp. 383-393. https://doi.org/10.1016/0950-5849(95)91494-K

[46] G. Ozsoyoglu, R. Snodgrass (1995). Temporal and real-time databases: a survey. IEEE Transactions on Knowledge and Data Engineering, vol. 7, no. 4, pp. 513-532. https://doi.org/10.1109/69.404027

[47] C. De Castro, F. Grandi, M.R. Scalas (1997). SCHEMA VERSIONING FOR MULTITEMPORAL RELATIONAL DATABASES. Information Systems, vol. 22, no. 5, pp. 249-290. https://doi.org/10.1016/S0306-4379(97)00017-3

[48] T.J. Green, G. Karvounarakis, V. Tannen (2007). Provenance semirings. Proceedings of the twenty-sixth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems, pp. 31-40. https://doi.org/10.1145/1265530.1265535

[49] Y.L. Simmhan, B. Plale, D. Gannon (2005). A survey of data provenance in e-science. ACM SIGMOD Record, vol. 34, no. 3, pp. 31-36. https://doi.org/10.1145/1084805.1084812

[50] L. Carata, S. Akoush, N. Balakrishnan, T. Bytheway, R. Sohan, M. Seltzer, et al. (2014). A primer on provenance. Communications of the ACM, vol. 57, no. 5, pp. 52-60. https://doi.org/10.1145/2596628

[51] B. Glavic (2021). Data Provenance. Foundations and Trends in Databases, vol. 9, no. 3-4, pp. 209-441. https://doi.org/10.1561/1900000068

[52] Y. Amsterdamer, D. Deutch, V. Tannen (2011). Provenance for aggregate queries. Proceedings of the thirtieth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems, pp. 153-164. https://doi.org/10.1145/1989284.1989302

[53] F. Jamil, A. Khan, A. Anjum, M. Ahmed, F. Jabeen, N. Javaid (2018). Secure provenance using an authenticated data structure approach. Computers & Security, vol. 73, pp. 34-56. https://doi.org/10.1016/j.cose.2017.10.005

[54] M.D. Wilkinson, M. Dumontier, I.J. Aalbersberg, G. Appleton, M. Axton, A. Baak, et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data, vol. 3, no. 1, article 160018. https://doi.org/10.1038/sdata.2016.18

[55] B. Pan, N. Stakhanova, S. Ray (2023). Data Provenance in Security and Privacy. ACM Computing Surveys, vol. 55, no. 14s, pp. 1-35. https://doi.org/10.1145/3593294

[56] L. Moreau, P. Groth, J. Cheney, T. Lebo, S. Miles (2015). The rationale of PROV. Journal of Web Semantics, vol. 35, pp. 235-257. https://doi.org/10.1016/j.websem.2015.04.001

[57] K. Wolstencroft, S. Owen, O. Krebs, Q. Nguyen, N.J. Stanford, M. Golebiewski, et al. (2015). SEEK: a systems biology data and model management platform. BMC Systems Biology, vol. 9, no. 1, article 33. https://doi.org/10.1186/s12918-015-0174-y

[58] D. Comer (1979). Ubiquitous B-Tree. ACM Computing Surveys, vol. 11, no. 2, pp. 121-137. https://doi.org/10.1145/356770.356776

[59] Y.S. Kwong, D. Wood (1980). On B-trees. Proceedings of the 1980 ACM SIGMOD international conference on Management of data - SIGMOD '80, pp. 207. https://doi.org/10.1145/582250.582282

[60] W.J. Hansen (1981). A Cost Model for the Internal Organization of B
⁺
-Tree Nodes. ACM Transactions on Programming Languages and Systems, vol. 3, no. 4, pp. 508-532. https://doi.org/10.1145/357146.357152

[61] M.A. Bender, E.D. Demaine, M. Farach-Colton (2005). Cache-Oblivious B-Trees. SIAM Journal on Computing, vol. 35, no. 2, pp. 341-358. https://doi.org/10.1137/S0097539701389956

[62] P. O'Neil, E. Cheng, D. Gawlick, E. O'Neil (1996). The log-structured merge-tree (LSM-tree). Acta Informatica, vol. 33, no. 4, pp. 351-385. https://doi.org/10.1007/s002360050048

[63] F. Chang, J. Dean, S. Ghemawat, W.C. Hsieh, D.A. Wallach, M. Burrows, et al. (2008). Bigtable. ACM Transactions on Computer Systems, vol. 26, no. 2, pp. 1-26. https://doi.org/10.1145/1365815.1365816

[64] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, et al. (2007). Dynamo. Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles, pp. 205-220. https://doi.org/10.1145/1294261.1294281

[65] M. Rosenblum, J.K. Ousterhout (1992). The design and implementation of a log-structured file system. ACM Transactions on Computer Systems, vol. 10, no. 1, pp. 26-52. https://doi.org/10.1145/146941.146943

[66] L. Lu, T.S. Pillai, H. Gopalakrishnan, A.C. Arpaci-Dusseau, R.H. Arpaci-Dusseau (2017). WiscKey. ACM Transactions on Storage, vol. 13, no. 1, pp. 1-28. https://doi.org/10.1145/3033273

[67] H.T. Vo, S. Wang, D. Agrawal, G. Chen, B.C. Ooi (2012). LogBase. Proceedings of the VLDB Endowment, vol. 5, no. 10, pp. 1004-1015. https://doi.org/10.14778/2336664.2336673

[68] C. Riegger, I. Petrov (2024). Storage Management with Multi-Version Partitioned BTrees. Information Systems, vol. 125, pp. 102403. https://doi.org/10.1016/j.is.2024.102403

APPENDIX A: REPRODUCIBILITY

The Revon GitHub repository provides the code and experiment records accompanying this paper. The paper/ directory holds the Linux reproducibility archive and the Windows reproducibility archive. Their README files explain how to set up dependencies, acquire public data, run experiments and verify the evidence [1]. Each bundle retains the sources it executed, its fixed protocol, raw measurements and internal audits.

The primary Linux, supplementary Linux and Windows campaigns follow different protocols, so they must be analysed separately. The documented preparation steps rebuild TLC inputs and generated repositories. Public-source metadata records licences, acquisition versions, checksums and the CO2 exclusion applied before timing. Run audits on disposable copies and localize the data paths to protect the collected records. Matching semantic hashes establishes consistency. It does not reproduce elapsed times or guarantee the same timing on another machine.

APPENDIX B: METHODS AND EVIDENCE DETAILS

The Linux bundles retain the executed sources and their hashes, environments, fixed workloads, execution order, raw observations and audits. Workers use PYTHONHASHSEED=20261008. Before timing begins, the primary campaign passes 67 unit tests and the supplementary campaign passes 69. Supplement preflight detects an incomplete transfer missing the test_data directory. We restore that empty fixture directory and pass the complete suite before taking measurements. The execution records retain the setup failure.

Every primary execution constructs a repository in a fresh worker. Model order is deterministically shuffled and rotated within repetition blocks. Import timing ends once the first durable version exists. For a history, incremental-commit latency is the median of its individual commit timers. Diff compares the first and last versions, returning sorted changed keys and old/new-value hashes as canonical JSON. Checkout emits the complete sorted key/value state as canonical UTF-8 JSON. Dolt timings include CLI launch, JSON parsing and output normalization, whereas Revon uses an in-process API. Workflow ratios therefore cannot be interpreted as engine-only speed comparisons.

We run correctness checks outside the operation timers. Each primary execution checks initial and final state hashes, every H+1 historical state, and the observed canonical diff against a separately built workload oracle. Verification records retain the observed and expected hashes. The audit regenerates workloads, checks matrix membership, selected paths and source/data/protocol identities, then recomputes the summaries. Downloaded

records are also audited locally. These checks establish internal integrity. They do not independently repeat the timing experiment.

We exclude warm-ups and summarize the seven measured repository trials using medians and interquartile ranges (IQRs). For latency ratios, we first pair matched trials, then calculate the median ratio and a paired percentile-bootstrap interval using 20,000 resamples. Ratios above one favour the denominator. The exploratory 95% intervals describe repeated timings of a fixed workload on one VM; they have no population or multiple-comparison interpretation. Taking the median of paired ratios can give a different result from dividing the two latency medians. We retain every completed trial, including timing outliers.

A parent process samples RSS, CPU and I/O for the worker and its descendants every 10 ms. Peaks include workload generation, oracle state, verification and storage work, so they cannot be attributed solely to the storage engine. Short-lived subprocesses and brief peaks may occur between samples. In the primary Linux sampler, an all-zero read counter is recorded as missing, preventing conclusions about read I/O from those records. The supplementary campaign distinguishes measured zero from unavailable counters and is analysed separately. Logical reads of cached data can occur while a process records zero physical read bytes.

Linux repository footprint is directory size before closing or compaction, including indexes, metadata and representation costs. The records also include normalization by logical payload and selected compaction probes on the first trial. They describe complete workflows rather than tree overhead in isolation. CPU, RSS and I/O span the worker lifecycle, which extends beyond the boundaries of individual operation timers.

The auxiliary Windows threshold experiment defines separate, non-overlapping calibration and validation profiles in advance. It tests operation-count candidates of 1,024, 2,048, 4,096, 8,192 and 16,384. Minimizing normalized calibration latency chooses T=16,384 before validation starts. On held-out profiles, its normalized score is 1.032, versus 1.297 for T=4,096. Always-log scores 1.000 in each split. These cached-query results belong to that experiment's recorded source and environment and are separate from Linux timings. A largest-candidate winner does not identify an optimum. Long-history and three-seed pilot records provide exploratory context.

The Linux threshold control fixes T=16,384 over 10,000 rows with 512 changes per commit. H=16 and H=64 produce 8,192 and 32,768 accumulated operations. Forced log, forced Merkle and hybrid use separate workers, with two warm-ups and seven measured trials each. Linux outcomes play no part in selecting the threshold. These cases test whether the auxiliary calibration choice transfers to Linux; they do not run another calibration/validation cycle.

For the Linux geometry workload, we use 10,000 rows, ten commits, 100 changes per commit and 32-byte synthetic values, with seed 20260824. We force Merkle diff for b4-d6, b8-d3, b8-d4, b8-d5 and b16-d3. Every configuration runs in fresh workers for two warm-ups and seven measured trials. Rotating the order limits confounding between time and configuration. Outside the timers, each execution checks eleven historical states and the endpoint diff. The adapter follows the workflow experiment's output contract. We report peaks from the geometry adapter's in-process monitor separately from external RSS measurements.

The Windows protocol and case manifest fix calibration seeds at 202610161/202610162 and use separate validation/sensitivity seeds 202610171-202610173. Workers use PYTHONHASHSEED=20261016. Monitoring uses seed 202610190, generated public corrections use 202610201-202610203, and scale uses 202610221-202610223. Queries within a worker share a repository, while each worker creates fresh persistent state. We

timestamp the threshold before validation and do not select it again afterwards. Linux/Windows rows remain separate.

Windows instrumentation uses Dolt's native JSON-diff command and the adapter's parser contract, with explicit timers for each phase. Preflight compares the actual output with the native adapter and the independent oracle. Outside timing, Revon queries verify observed output hashes, all historical-state hashes, reverse/identity requests, object integrity and reopening. We document setup checks and data-quality exclusions and apply them before measurement. The retained query matrix and timing-source hashes record the execution conditions.

The portable audit works on a disposable copy of the evidence. It starts a fresh interpreter, imports the executed source and localizes the recorded public-data path. It then regenerates workloads and state/diff oracles, checks every configuration, trial and order identity, and recomputes numerical summaries. Finally, it verifies the hashes of the collected files. The archives supply these checks and the data-acquisition recipes to verify consistency. Reproducing timings requires running the experiments.

Table 16
Checksums for public data and the Linux protocol

Item	Definition
Source	NYC TLC yellow_tripdata_2024-01.parquet; 2,964,624 source rows
Raw SHA-256	c4d59da7bbc8abaeeeb1727947ee93d9891a71acb42854bd80db1571b2030510
Prepared million-row SHA-256	6608b7680e82432ca22f90a71d4994676ffd290c4fc85f17f91fab03d4c7b661
P1	100,000 records; H=10; C=100; 2 warm-ups + 7 measured trials per system
Scale diagnostic	1,000,000 records; H=10; C=1,000; one trial per system; no interval or ranking

Table 17
Windows execution matrix; cached queries count separately from independent history or repository repetitions

Component	Fresh workers	Histories / control	Warm-up queries	Measured queries
Calibration	12	2 seeds × 2 localities × 3 densities	144	504
Held-out	18	3 different seeds × same profiles	216	756
Sensitivity	21	3 seeds × 7 matched variations	252	882
Monitor diagnostic	6	3 off/on pairs, one fixed history	72	252
Public sources	18	2 sources × 3 runs × M/H/Dolt	36	126
Scale	18	2 sizes × 3 seeds × M/H/Dolt	36	126
Total	93	Different campaigns are never pooled	756	2646